



**Universidad
Andrés Bello®**

UNIVERSIDAD ANDRÉS BELLO

Facultad de Ingeniería

Sistema de Visión Artificial e Integración Industrial

**Detección de Defectos en Snacks mediante YOLOv11,
Node-RED y PLC S7-1200**

Integrantes:

César Laureano, Gaspar Yañes, Sigfrid Ureta, Luis Morales y Fernando Venegas

Santiago, Chile
30 de junio de 2026

Índice

1. Introducción	2
2. Diccionario de Variables Implementado	2
2.1. Matriz de Permisos de Escritura	2
3. Marco Teórico: Redes Neuronales Convolucionales (RNA Co.)	3
4. Metodología y Desarrollo	3
4.1. Procesamiento de Datos y Entrenamiento IA	3
4.2. Iteraciones de Captura de Video (Troubleshooting)	4
4.3. Script de Inferencia y Comunicación Local (Node-RED)	5
5. Arquitectura de Integración (Node-RED y PLC)	7
5.1. Flujo de Comunicación en Node-RED	7
5.2. Lógica de Control en TIA Portal (PLC S7-1200)	9
5.3. Supervisión HMI (KTP600 Basic)	10
6. Conclusiones	10

1. Introducción

El presente proyecto tiene como objetivo desarrollar e implementar un sistema de inspección óptica automatizado capaz de identificar papas fritas con defectos de sobre-cocción (quemaduras) en un flujo de procesamiento continuo.

El alcance del trabajo abarca desde la captura y etiquetado de un dataset propio, el entrenamiento de una red neuronal convolucional (RNA Co.), hasta la integración de las inferencias con hardware industrial (OT). Para lograr esto, se estableció una arquitectura de comunicación en la que un script de Python interactúa mediante el protocolo HTTP con Node-RED, el cual a su vez escribe variables en un PLC Siemens S7-1200 que gobierna la lógica de rechazo, supervisado mediante una pantalla HMI KTP600.

2. Diccionario de Variables Implementado

Aunque la planificación inicial contemplaba múltiples métricas de conteo y porcentajes estadísticos, durante la fase de integración IT/OT el alcance se optimizó para procesar exclusivamente el objetivo principal: la detección binaria de calidad y su consecuente acción de rechazo.

Por lo tanto, el diccionario de variables y la comunicación en red se simplificaron a los dos tags esenciales mostrados a continuación, reduciendo la carga de datos en el controlador y eliminando latencias innecesarias.

Tag de Variable	Tipo	Origen	Uso y Aplicación en el Sistema
PAPA_QUEMADA	BOOL	Python	Señal de estado proveniente de la inferencia óptica. Se escribe vía Node-RED en la marca de memoria %M0.0 del PLC. 1 = Unidad QUEMADA, 0 = Unidad NORMAL.
OUT_Q0	BOOL	PLC	Estado de la salida (%Q0.0) del PLC. Se utiliza de forma exclusiva como marca de enlace para gatillar la animación dinámica de alerta en la pantalla HMI.

Cuadro 1: Diccionario de variables simplificado según la implementación final.

2.1. Matriz de Permisos de Escritura

Se mantuvieron los siguientes privilegios exclusivos para la modificación de datos en la red local para evitar colisiones:

- **Python (vía Node-RED)** escribe con exclusividad sobre el tag de alerta: PAPA_QUEMADA.
- **TIA Portal** determina de manera exclusiva la activación del bit de salida física: OUT_Q0, replicando este estado hacia la lectura del HMI.

3. Marco Teórico: Redes Neuronales Convolucionales (RNA Co.)

Para el desarrollo de este proyecto de inspección óptica, se emplea exclusivamente la rama de las **RNA Co.** (Redes Neuronales Convolucionales). Estos modelos están enfocados estrictamente en el procesamiento de imágenes y visión computacional, proporcionando las herramientas de matriz espacial indispensables para ejecutar la segmentación de defectos en los snacks. Específicamente, se utiliza la arquitectura **YOLOv11**, la cual permite la detección de objetos y clasificación de defectos en flujos de video de forma veloz y continua.

4. Metodología y Desarrollo

4.1. Procesamiento de Datos y Entrenamiento IA

El proyecto inició con la captura manual de 400 fotografías de muestras de papas fritas, tanto en estado óptimo como con presencia de quemaduras. Este dataset fue subido, pre-procesado y segmentado utilizando la plataforma **Roboflow**. Posteriormente, se exportó el modelo para ser entrenado bajo la arquitectura **YOLOv11**.

A continuación, se detalla el bloque de código fundamental utilizado en el entorno local (Jupyter Notebook) para la importación del dataset y la configuración de los hiperparámetros de entrenamiento utilizando aceleración por hardware (GPU):

```
1 import torch
2 from roboflow import Roboflow
3 from ultralytics import YOLO
4
5 # 1. Verificación de aceleración por hardware (GPU)
6 if torch.cuda.is_available():
7     print(f"GPU Detectada y Lista: {torch.cuda.get_device_name(0)}")
8
9 # 2. Descarga del conjunto de datos desde Roboflow
10 rf = Roboflow(api_key="TU_API_KEY")
11 project = rf.workspace("g-yezgmez-uandresbello-edu").project("papas-cag92")
12 dataset = project.version(1).download("yolov11")
13
14 # 3. Configuración y Entrenamiento del Modelo
15 model = YOLO('yolo11n.pt') # Arquitectura base pre-entrenada
16
17 resultados = model.train(
18     data=f"{dataset.location}/data.yaml",
19     epochs=50,                # Iteraciones completas
20     imgsz=640,               # Resolución de análisis
21     batch=16,                # Tamaño de lote ajustado a la VRAM
22     device=0,                # Identificador de GPU NVIDIA
23     project='Papas-Quemadas',
24     plots=True                # Generación de métricas de rendimiento
```

Listing 1: Verificación de Entorno y Entrenamiento del Modelo

4.2. Iteraciones de Captura de Video (Troubleshooting)

Para la captura de video de forma continua, se evaluaron distintas alternativas buscando optimizar el rendimiento y la latencia computacional. Cabe destacar que la decisión de obviar el uso del microcontrolador ESP32-CAM y optar por periféricos locales fue consultada y autorizada expresamente por el profesor de la asignatura, con el fin de avanzar con opciones que aseguraran la factibilidad técnica del control industrial. Las pruebas fueron:

1. **Cámara vía DroidCam (WiFi):** Se intentó utilizar un teléfono móvil como cámara IP periférica. Esta opción fue descartada debido a una alta latencia en la transmisión de red, lo que impedía un control de rechazo preciso.
2. **Procesamiento Local en Celular (Termux):** Se configuró un entorno Linux (Termux) en un Samsung S25 para ejecutar el modelo directamente en el borde (Edge Computing) y enviar los datos a Node-RED. Esta alternativa se descartó por incompatibilidad de librerías específicas de YOLO con el entorno ARM del dispositivo.
3. **Cámara Externa de Alta Resolución (Solución Final):** Se optó por una cámara externa proporcionada por la universidad, conectada directamente al equipo de procesamiento. Esto eliminó la latencia y permitió un flujo continuo y estable.

4.3. Script de Inferencia y Comunicación Local (Node-RED)

En este bloque se realiza la captura de video y se envía la alerta booleana mediante una petición HTTP (POST) hacia el entorno de Node-RED basándose en las clasificaciones preestablecidas ("NORMAL" o "QUEMADA").

```

1 # 4. Deteccion Continua y Conexion con PLC (Node-RED)
2 import cv2
3 import os
4 import requests # Libreria para enviar datos a Node-RED
5 from ultralytics import YOLO
6
7 print("\n--- DETECCION CONTINUA Y COMUNICACION INDUSTRIAL ---")
8
9 # Configuracion de Rutas y Red
10 ruta_mejores_pesos = r"C:\Users\sieg\Downloads\Modelo Entrenado\Modelo
    Entrenado\best.pt"
11 fuente_video = 0
12 NODE_RED_URL = "http://127.0.0.1:1880/recibir-vision"
13
14 if not os.path.exists(ruta_mejores_pesos):
15     raise FileNotFoundError(f"[ERROR]: No se encontro el archivo:\n{
        ruta_mejores_pesos}")
16
17 print(f"[OK] Cargando modelo YOLO desde:\n{ruta_mejores_pesos}")
18 modelo_continuo = YOLO(ruta_mejores_pesos)
19
20 cap = cv2.VideoCapture(fuente_video)
21
22 if cap.isOpened():
23     print("[OK] Camara conectada con exito!")
24     estado_anterior = None
25
26     while True:
27         ret, frame = cap.read()
28         if not ret:
29             break
30
31         # Prediccion sobre el cuadro actual
32         resultados = modelo_continuo(frame, stream=True, conf=0.45,
            verbose=False)
33         hay_quemada = False
34
35         for r in resultados:
36             frame_annotado = r.plot()
37             cv2.imshow("Detector de Papas", frame_annotado)
38
39             # Analizar detecciones de la red neuronal
40             for cls_id in r boxes.cls:
41                 nombre_clase = modelo_continuo.names[int(cls_id)]
42                 if nombre_clase == "quemada":
43                     ESTADO = "QUEMADA"
44                     hay_quemada = True
45                     break

```

```
46         else:
47             ESTADO = "NORMAL"
48
49     # Envio de senal al PLC via Node-RED
50     if hay_quemada != estado_anterior:
51         payload = {"valor": hay_quemada}
52         try:
53             requests.post(NODE_RED_URL, json=payload, timeout=0.5)
54             if hay_quemada:
55                 print(f"[ALERTA] Estado: {ESTADO} -> PLC: True (1)")
56             else:
57                 print(f"[INFO] Estado: {ESTADO} -> PLC: False (0)")
58         except requests.exceptions.RequestException:
59             print("[ADVERTENCIA] No se pudo conectar con Node-RED.")
60
61         estado_anterior = hay_quemada
62
63     if cv2.waitKey(1) & 0xFF == ord('q'):
64         break
65
66 cap.release()
67 cv2.destroyAllWindows()
68 cv2.waitKey(1)
```

Listing 2: Detección Continua y Conexión con PLC vía Node-RED

Para la ingesta de datos, se configuró un nodo **HTTP In** operando en la ruta local `/recibir-vision`. Posteriormente, la carga útil se dirige a un nodo **S7 Out** configurado para apuntar a la dirección IP del PLC (10.41.32.17), escribiendo directamente sobre la marca de memoria M0.0.

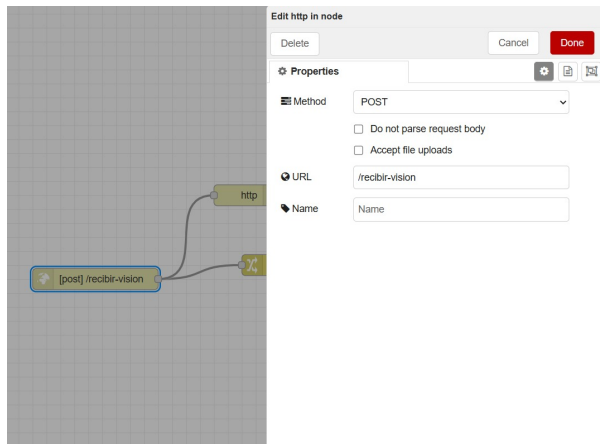


Figura 2: Nodo HTTP In.

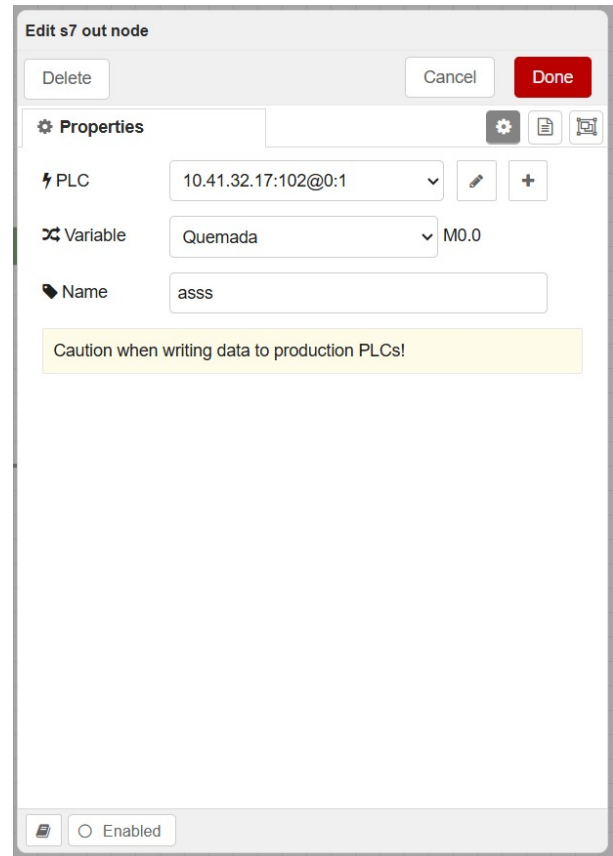


Figura 3: Nodo S7 Out hacia el PLC.

5.2. Lógica de Control en TIA Portal (PLC S7-1200)

El autómata programable Siemens S7-1200 fue configurado dentro de la misma subred, asignándole la dirección IP estática 10.41.32.17 a través de su interfaz PROFINET.

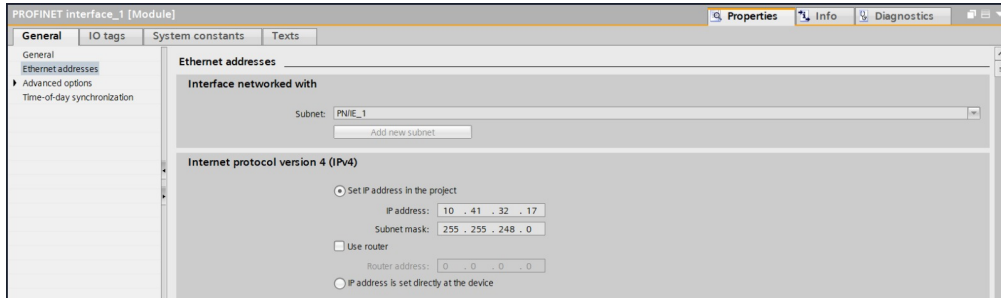


Figura 4: Configuración de dirección IP del PLC S7-1200 en TIA Portal.

El equipo recibe la alerta desde Node-RED en la marca de memoria %M0.0 (PAPA_QUEMADA). La lógica en lenguaje Ladder (OB1) establece que, al cerrarse este contacto, se energiza la salida %Q0.0. En esta arquitectura optimizada, dicha salida se utiliza estrictamente de manera virtual para procesar el evento de rechazo y enviar la señal de actualización visual a la interfaz del operador, sin accionar componentes mecánicos físicos.

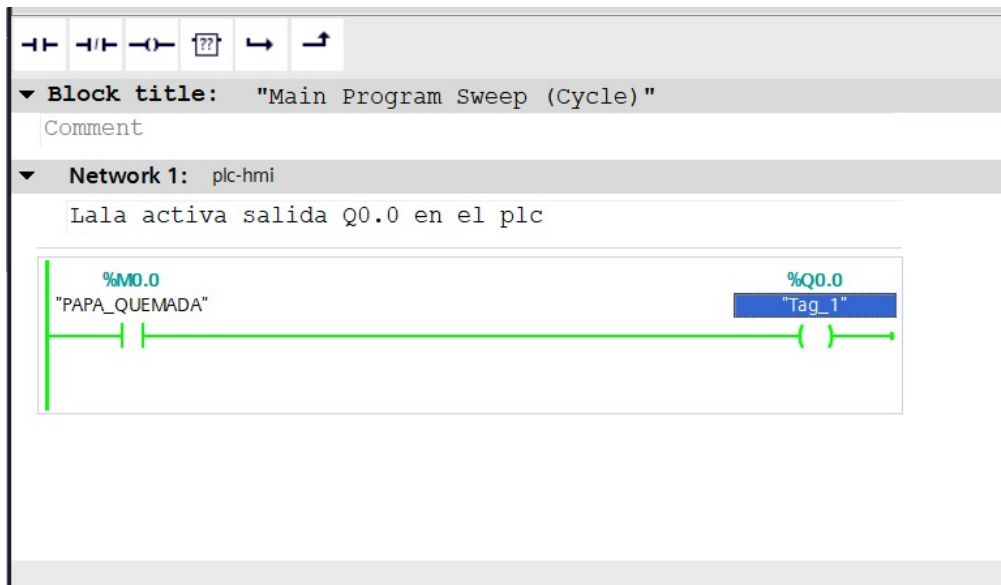


Figura 5: Lógica Ladder en TIA Portal activando la salida Q0.0 para enlace visual.

5.3. Supervisión HMI (KTP600 Basic)

Como redundancia, se programó una pantalla HMI KTP600 Basic Color PN. Para asegurar la comunicación con el PLC, se le asignó la dirección IP 10.41.32.22. En la pantalla principal, titulada "Papas Bakanes", se incluyó un indicador gráfico que cambia dinámicamente de color al detectar una unidad defectuosa (ESTADO = "QUEMADA").

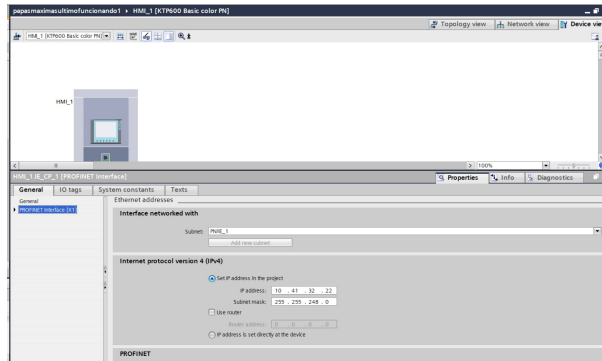


Figura 6: Dirección IP del panel HMI.

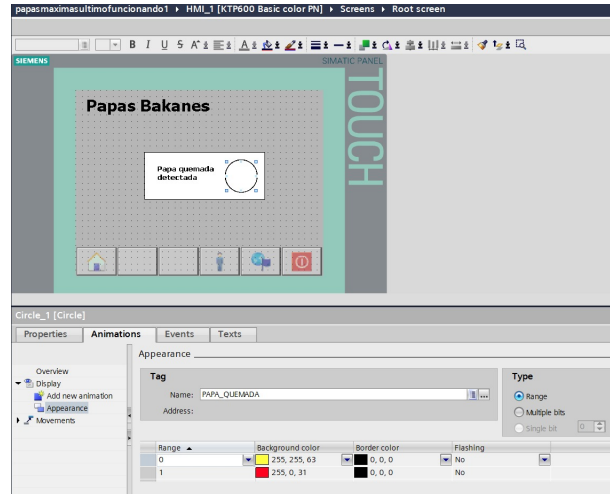


Figura 7: Interfaz HMI con animación dinámica.

6. Conclusiones

El proyecto demostró con éxito la viabilidad de integrar herramientas modernas de Inteligencia Artificial (YOLOv11, Python) con hardware de control industrial tradicional (S7-1200, HMI KTP600) operando como puente el entorno Node-RED de forma ininterrumpida.

La metodología de ensayo y error aplicada para la captura de video resultó vital. Contar con la autorización para flexibilizar el uso de microcontroladores periféricos permitió descartar soluciones incompatibles a favor de una captura directa de alta resolución, garantizando una respuesta precisa para la actuación de las salidas del PLC ante estados de alerta.